SNES Assembly Adventure 08: Reading and Handling Input

24 Feb 2022

Welcome back, Adventurer! Last time, we learned more about direct memory access and moving sprites.

This time, we'll read the joypad and move the sprites accordingly. So this time we will

- read and process input from the joypad
- make the sprites move according to input
- implement some simple collision detection to make the sprites stay on screen



You earned the subroutine badge!

Without further ado, let's get started!

Preparing The Demo

For this demo, we're going to reuse a lot of the code from the bouncing sprite demo from last time.

Copy all the code, sprite, and color data from that demo as we'll be reusing them here. You can find all source code and other files in the SNES Assembly Adventure repository on Github.

Reading The Joypad

As always, I'll show you the code first, then we'll discuss it line by line:

```
1   ; ------------------------------------------------------------------------
2   ;   File: JoypadSprite.s
3   ;   Description: Displays a sprite that bounces off the edges
4   ; ------------------------------------------------------------------------
5
6   ;----- Aliases/Labels ----------------------------------------------------
7   ; these are aliases for the Memory Mapped Registers we will use
8   INIDISP     = $2100     ; inital settings for screen
9   OBJSEL      = $2101     ; object size $ object data area designation
10  OAMADDL     = $2102     ; address for accessing OAM
11  OAMADDH     = $2103
12  OAMDATA     = $2104     ; data for OAM write
13  VMAINC      = $2115     ; VRAM address increment value designation
14  VMADDL      = $2116     ; address for VRAM read and write
15  VMADDH      = $2117
16  VMDATAL     = $2118     ; data for VRAM write
17  VMDATAH     = $2119
18  CGADD       = $2121     ; address for CGRAM read and write
19  CGDATA      = $2122     ; data for CGRAM write
20  TM          = $212c     ; main screen designation
21  HVBJOY      = $4212     ; H/V blank flags and standard controller enable flag
22  STDCNTRL1L  = $4218     ; data for standard controller I
23  STDCNTRL1H  = $4219
24  NMITIMEN    = $4200     ; enable flaog for v-blank
25  MDMAEN      = $420b     ; DMA enable register
26  RDNMI       = $4210     ; read the NMI flag status
27  DMAP0       = $4300     ; DMA control register, channel 0
28  BBAD0       = $4301     ; DMA destination register, channel 0
29  A1T0L       = $4302     ; DMA source address register low, channel 0
```

```
30  A1T0H       = $4303     ; DMA source address register high, channel 0
31  A1T0B       = $4304     ; DMA source address register bank, channel 0
32  DAS0L       = $4305     ; DMA size register low, channel 0
33  DAS0H       = $4306     ; DMA size register high, channel 0
34  ;-------------------------------------------------------------------------
35
36  ;----- Memory Map WRAM ---------------------------------------------------
37  HOR_SPEED   = $0300     ; the horizontal speed
38  VER_SPEED   = $0301     ; the vertical speed
39  JOYPAD1     = $0302     ; data read from joypad 1
40  JOYTRIGGER1 = $0304     ; trigger read from joypad 1
41  JOYHELD1    = $0306     ; held buttons read from joypad 1
42  OAMMIRROR   = $0400     ; location of OAMRAM mirror in WRAM
43  ;-------------------------------------------------------------------------
44
45  ;----- Game Constants ----------------------------------------------------
46      ; we use these constants to check for collisions with the screen boundarie
47  SCREEN_LEFT     = $00   ; left screen boundary = 0
48  SCREEN_RIGHT    = $ff   ; right screen boundary = 255
49  SCREEN_TOP      = $00   ; top screen boundary = 0
50  SCREEN_BOTTOM   = $df   ; bottom screen boundary = 223
51      ; a simple constant to define the sprite movement speed
52  SPRITE_SPEED    = $02   ; the sprites will move 2 pixel per frame
53      ; this makes the code a bit more readable
54  SPRITE_SIZE     = $08   ; sprites are 8 by 8 pixel
55  OAMMIRROR_SIZE  = $0220 ; OAMRAM can hold data for 128 sprites, 4 bytes each
56      ; constants to use as masks
57  UP_BUTTON       = $0800
58  DOWN_BUTTON     = $0400
59  LEFT_BUTTON     = $0200
60  RIGHT_BUTTON    = $0100
61  ;-------------------------------------------------------------------------
62
63  ;----- Assembler Directives ----------------------------------------------
64  .p816                       ; tell the assembler this is 65816 code
65  ;-------------------------------------------------------------------------
66
67  ;----- Includes ----------------------------------------------------------
68  .segment "SPRITEDATA"
69  SpriteData: .incbin "Sprites.vra"
70  ColorData:  .incbin "SpriteColors.pal"
71  ;-------------------------------------------------------------------------
72
73  .segment "CODE"
```

```
74  ;---------------------------------------------------------------------------
75  ;   This is the entry point of the demo
76  ;---------------------------------------------------------------------------
77  .proc   ResetHandler
78          sei                     ; disable interrupts
79          clc                     ; clear the carry flag
80          xce                     ; switch the 65816 to native (16-bit mode)
81          rep #$10                ; set X and Y to 16-bit
82          sep #$20                ; set A to 8-bit
83          lda #$8f                ; force v-blanking
84          sta INIDISP
85          stz NMITIMEN            ; disable NMI
86          ; set the stack pointer to $1fff
87          ldx #$1fff              ; load X with $1fff
88          txs                     ; copy X to stack pointer
89
90          ; load sprites into VRAM
91          tsx                     ; save current stack pointer
92          pea $0000               ; push VRAM destination address to stack
93          pea SpriteData          ; push sprite data source address to stack
94          lda #$80                ; number of bytes (128/$80) to transfer
95          pha
96          jsr LoadVRAM            ; transfer sprite data to VRAM
97          txs                     ; restore old stack pointer
98
99          ; load color data into CGRAM
100         tsx                     ; save current stack pointer
101         lda #$80                ; destination address in CGRAM
102         pha
103         pea ColorData           ; color data source address
104         lda #$20                ; number of bytes (32/$20) to transfer
105         pha
106         jsr LoadCGRAM           ; transfer color data into CGRAM
107         txs                     ; restore old stack pointer
108
109         ; initialize OAMRAM mirror
110         ldx #$00
111         ; upper-left sprite
112         lda #(SCREEN_RIGHT/2 - SPRITE_SIZE) ; sprite 1, horizontal position
113         sta OAMMIRROR, X
114         inx                                 ; increment index
115         lda #(SCREEN_BOTTOM/2 - SPRITE_SIZE); sprite 1, vertical position
116         sta OAMMIRROR, X
117         inx
```

```
118         lda #$00                            ; sprite 1, name
119         sta OAMMIRROR, X
120         inx
121         lda #$00                            ; no flip, palette 0
122         sta OAMMIRROR, X
123         inx
124         ; upper-right sprite
125         lda #(SCREEN_RIGHT/2)               ; sprite 3, horizontal position
126         sta OAMMIRROR, X
127         inx                                 ; increment index
128         lda #(SCREEN_BOTTOM/2 - SPRITE_SIZE); sprite 3, vertical position
129         sta OAMMIRROR, X
130         inx
131         lda #$01                            ; sprite 3, name
132         sta OAMMIRROR, X
133         inx
134         lda #$00                            ; no flip, palette 0
135         sta OAMMIRROR, X
136         inx
137         ; lower-left sprite
138         lda #(SCREEN_RIGHT/2 - SPRITE_SIZE) ; sprite 2, horizontal position
139         sta OAMMIRROR, X
140         inx                                 ; increment index
141         lda #(SCREEN_BOTTOM/2)              ; sprite 2, vertical position
142         sta OAMMIRROR, X
143         inx
144         lda #$02                            ; sprite 2, name
145         sta OAMMIRROR, X
146         inx
147         lda #$00                            ; no flip, palette 0
148         sta OAMMIRROR, X
149         inx
150         ; lower-right sprite
151         lda #(SCREEN_RIGHT/2)                ; sprite 4, horizontal position
152         sta OAMMIRROR, X
153         inx                                 ; increment index
154         lda #(SCREEN_BOTTOM/2)              ; sprite 4, vertical position
155         sta OAMMIRROR, X
156         inx
157         lda #$03                            ; sprite 4, name
158         sta OAMMIRROR, X
159         inx
160         lda #$00                            ; no flip, palette 0
161         sta OAMMIRROR, X
```

```
162         inx
163         ; move the other sprites off screen
164         rep #$20                        ; set A to 16-bit
165         lda #$f180                      ; Y = 241, X = -128
166 OAMLoop:
167         sta OAMMIRROR, X
168         inx
169         inx
170         cpx #(OAMMIRROR_SIZE - $20)
171         bne OAMLoop
172         ; correct bit 9 of horizontal/X position, set size to 8x8
173         lda #$5555
174 OBJLoop:
175         sta OAMMIRROR, X
176         inx
177         inx
178         cpx #OAMMIRROR_SIZE
179         bne OBJLoop
180
181         sep #$20                        ; set A to 8-bit
182         ; correct extra OAM byte for first four sprites
183         ldx #$0200
184         lda #$00
185         sta OAMMIRROR, X
186
187         ; set initial horizontal and vertical speed
188         lda #SPRITE_SPEED
189         sta HOR_SPEED
190         sta VER_SPEED
191
192         ; make Objects visible
193         lda #$10
194         sta TM
195         ; release forced blanking, set screen to full brightness
196         lda #$0f
197         sta INIDISP
198         ; enable NMI, turn on automatic joypad polling
199         lda #$81
200         sta NMITIMEN
201
202         jmp GameLoop            ; all initialization is done
203 .endproc
204 ;---------------------------------------------------------------------------
205
```

```
206 ;---------------------------------------------------------------------------
207 ;   Executed during V-Blank
208 ;---------------------------------------------------------------------------
209 .proc   GameLoop
210         wai                                 ; wait for NMI / V-Blank
211
212         ; read joypad 1
213         ; check whether joypad is ready
214 WaitForJoypad:
215         lda HVBJOY                          ; get joypad status
216         and #$01                            ; check whether joypad done readin
217         beq WaitForJoypad                   ; ...if not, wait a bit more
218         ; first, check for newly pressed buttons since last frame
219         rep #$20                            ; set A to 16-bit
220         lda STDCNTRL1L                      ; get new input from this frame
221         ldy JOYPAD1                         ; get input from last frame
222         sta JOYPAD1                         ; store new input from this frame
223         tya                                 ; check for newly pressed buttons.
224         eor JOYPAD1                         ; filter buttons that were not pre
225         and JOYPAD1                         ; filter held buttons from last fr
226         sta JOYTRIGGER1                     ; ...and store them
227         ; second, check for buttons held from last frame
228         tya                                 ; get input from last frame
229         and JOYPAD1                         ; filter held buttons from last fr
230         sta JOYHELD1                        ; ...store them
231
232         ; check the dpad, if any of the directional buttons where pressed or h
233         ; move the sprites accordingly
234 CheckUpButton:
235         lda #$0000                          ; set A to zero
236         ora JOYTRIGGER1                     ; check whether the up button was
237         ora JOYHELD1                        ; ...or held from last frame
238         and #UP_BUTTON
239         beq CheckUpButtonDone               ; if neither has occured, move on
240         ; else, move sprites up
241         ldy #$0000                          ; Y is the loop counter
242         ldx #$0001                          ; set offset to 1, to manipulate s
243         sep #$20                            ; set A to 8-bit
244 MoveSpritesUp:
245         lda OAMMIRROR, X
246         sec
247         sbc VER_SPEED
248         bcc CorrectVerticalPositionDown     ; if vertical position is below ze
249         sta OAMMIRROR, X
```

```
250         inx                             ; increment X by 4
251         inx
252         inx
253         inx
254         iny
255         cpy #$0004                      ; unless Y = 4, continue loop
256         bne MoveSpritesUp
257 CheckUpButtonDone:
258         rep #$20                        ; set A to 16-bit
259
260 CheckDownButton:
261         lda #$0000                      ; set A to zero
262         ora JOYTRIGGER1                 ; check whether the down button wa
263         ora JOYHELD1                    ; ...or held from last frame
264         and #DOWN_BUTTON
265         beq CheckDownButtonDone         ; if neither has occured, move on
266         ; else, move sprites down
267         ldy #$0000                      ; Y is the loop counter
268         ldx #$0001                      ; set offset to 1, to manipulate s
269         sep #$20                        ; set A to 8-bit
270         ; check if sprites move below buttom boundry
271         lda OAMMIRROR, X
272         clc
273         adc VER_SPEED
274         cmp #(SCREEN_BOTTOM - 2 * SPRITE_SIZE)
275         bcs CorrectVerticalPositionUp
276 MoveSpritesDown:
277         lda OAMMIRROR, X
278         clc
279         adc VER_SPEED
280         sta OAMMIRROR, X
281         inx                             ; increment X by 4
282         inx
283         inx
284         inx
285         iny
286         cpy #$0004                      ; unless Y = 4, continue loop
287         bne MoveSpritesDown
288 CheckDownButtonDone:
289         rep #$20                        ; set A to 16-bit
290         jmp CheckLeftButton             ; continue input check
291
292 CorrectVerticalPositionDown:
293         sep #$20                        ; set A to 8-bit
```

```
294         stz OAMMIRROR + 1                   ; sprite 1, vertical position
295         stz OAMMIRROR + 5                   ; sprite 3, vertical position
296         lda #SPRITE_SIZE
297         sta OAMMIRROR + 9                   ; sprite 2, vertical position
298         sta OAMMIRROR + 13                  ; sprite 4, vertical position
299         rep #$20                            ; set A to 16-bit
300         jmp CheckLeftButton                 ; continue input check
301
302 CorrectVerticalPositionUp:
303         sep #$20                            ; set A to 8-bit
304         lda #(SCREEN_BOTTOM - 2 * SPRITE_SIZE)
305         sta OAMMIRROR + 1                   ; sprite 1, vertical position
306         sta OAMMIRROR + 5                   ; sprite 3, vertical position
307         lda #(SCREEN_BOTTOM - SPRITE_SIZE)
308         sta OAMMIRROR + 9                   ; sprite 2, vertical position
309         sta OAMMIRROR + 13                  ; sprite 4, vertical position
310         rep #$20                            ; set A to 16-bit
311
312 CheckLeftButton:
313         lda #$0000                          ; set A to zero
314         ora JOYTRIGGER1                     ; check whether the up button was
315         ora JOYHELD1                        ; ...or held from last frame
316         and #LEFT_BUTTON
317         beq CheckLeftButtonDone             ; if neither has occured, move on
318         ; else, move sprites up
319         ldy #$0000                          ; Y is the loop counter
320         ldx #$0000                          ; set offset to 0, to manipulate s
321         sep #$20                            ; set A to 8-bit
322 MoveSpritesLeft:
323         lda OAMMIRROR, X
324         sec
325         sbc HOR_SPEED
326         bcc CorrectHorizontalPositionRight
327         sta OAMMIRROR, X
328         inx                                 ; increment X by 4
329         inx
330         inx
331         inx
332         iny
333         cpy #$0004                          ; unless Y = 4, continue loop
334         bne MoveSpritesLeft
335 CheckLeftButtonDone:
336         rep #$20                            ; set A to 16-bit
337
```

```
338 CheckRightButton:
339         lda #$0000                      ; set A to zero
340         ora JOYTRIGGER1                 ; check whether the down button wa
341         ora JOYHELD1                    ; ...or held from last frame
342         and #RIGHT_BUTTON
343         beq CheckRightButtonDone        ; if neither has occured, move on
344         ; else, move sprites down
345         ldy #$0000                      ; Y is the loop counter
346         ldx #$0000                      ; set offset to 0, to manipulate s
347         sep #$20                        ; set A to 8-bit
348         ; check whether sprites move beyond right boundry
349         lda OAMMIRROR, X
350         clc
351         adc HOR_SPEED
352         cmp #(SCREEN_RIGHT - 2 * SPRITE_SIZE)
353         bcs CorrectHorizontalPositionLeft
354 MoveSpritesRight:
355         lda OAMMIRROR, X
356         clc
357         adc HOR_SPEED
358         sta OAMMIRROR, X
359         inx                             ; increment X by 4
360         inx
361         inx
362         inx
363         iny
364         cpy #$0004                      ; unless Y = 4, continue loop
365         bne MoveSpritesRight
366 CheckRightButtonDone:
367         rep #$20                        ; set A to 16-bit
368         jmp InputDone
369
370 CorrectHorizontalPositionRight:
371         sep #$20                        ; set A to 8-bit
372         stz OAMMIRROR + 0               ; sprite 1, horizontal position
373         stz OAMMIRROR + 8               ; sprite 2, horizontal position
374         lda #SPRITE_SIZE
375         sta OAMMIRROR + 4               ; sprite 3, horizontal position
376         sta OAMMIRROR + 12              ; sprite 4, horizontal position
377         jmp InputDone
378
379 CorrectHorizontalPositionLeft:
380         sep #$20                        ; set A to 8-bit
381         lda #(SCREEN_RIGHT - 2 * SPRITE_SIZE)
```

```
382         sta OAMMIRROR + 0                   ; sprite 1, horizontal position
383         sta OAMMIRROR + 8                   ; sprite 2, horizontal position
384         lda #(SCREEN_RIGHT - SPRITE_SIZE)
385         sta OAMMIRROR + 4                   ; sprite 3, horizontal position
386         sta OAMMIRROR + 12                  ; sprite 4, horizontal position
387
388 InputDone:
389         sep #$20                            ; set A back to 8-bit
390
391         jmp GameLoop
392 .endproc
393 ;-------------------------------------------------------------------------
394
395 ;-------------------------------------------------------------------------
396 ;   Will be called during V-Blank every frame
397 ;-------------------------------------------------------------------------
398 .proc   NMIHandler
399         lda RDNMI               ; read NMI status, acknowledge NMI
400
401         tsx                     ; save old stack pointer
402         pea OAMMIRROR           ; push mirror address to stack
403         jsr UpdateOAMRAM        ; update OAMRAM
404         txs                     ; restore old stack pointer
405
406         rti
407 .endproc
408 ;-------------------------------------------------------------------------
409
410 ;-------------------------------------------------------------------------
411 ;   Is not used in this program
412 ;-------------------------------------------------------------------------
413 .proc   IRQHandler
414         ; code
415         rti
416 .endproc
417 ;-------------------------------------------------------------------------
418
419 ;-------------------------------------------------------------------------
420 ;   Load sprite data into VRAM
421 ;   Parameters: NumBytes: .byte, SrcPointer: .addr, DestPointer: .addr
422 ;-------------------------------------------------------------------------
423 .proc   LoadVRAM
424         phx                     ; save old stack pointer
425         ; create frame pointer
```

```
426         phd                     ; push Direct Register to stack
427         tsc                     ; transfer Stack to... (via Accumulator)
428         tcd                     ; ...Direct Register.
429         ; use constants to access arguments on stack with Direct Addressing
430         NumBytes    = $07       ; number of bytes to transfer
431         SrcPointer  = $08       ; source address of sprite data
432         DestPointer = $0a       ; destination address in VRAM
433
434         ; set destination address in VRAM, and address increment after writing
435         ldx DestPointer         ; load the destination pointer...
436         stx VMADDL              ; ...and set VRAM address register to it
437         lda #$80
438         sta VMAINC              ; increment VRAM address by 1 when writing to
439
440         ; loop through source data and transfer to VRAM
441         ldy #$0000              ; set register Y to zero, we will use Y as a l
442 VRAMLoop:
443         lda (SrcPointer, S), Y  ; get bitplane 0/2 byte from the sprite data
444         sta VMDATAL             ; write the byte in A to VRAM
445         iny                     ; increment counter/offset
446         lda (SrcPointer, S), Y  ; get bitplane 1/3 byte from the sprite data
447         sta VMDATAH             ; write the byte in A to VRAM
448         iny                     ; increment counter/offset
449         cpy NumBytes            ; check whether we have written $04 * $20 = $8
450         bcc VRAMLoop            ; if X is smaller than $80, continue the loop
451
452         ; all done
453         pld                     ; restore caller's frame pointer
454         plx                     ; restore old stack pointer
455         rts
456 .endproc
457 ;-------------------------------------------------------------------------------
458
459 ;-------------------------------------------------------------------------------
460 ;   Load color data into CGRAM
461 ;   NumBytes: .byte, SrcPointer: .byte, DestPointer: .addr
462 ;-------------------------------------------------------------------------------
463 .proc   LoadCGRAM
464         phx                     ; save old stack pointer
465         ; create frame pointer
466         phd                     ; push Direct Register to stack
467         tsc                     ; transfer Stack to... (via Accumulator)
468         tcd                     ; ...Direct Register.
469         ; use constants to access arguments on stack with Direct Addressing
```

```
470         NumBytes    = $07       ; number of bytes to transfer
471         SrcPointer  = $08       ; source address of sprite data
472         DestPointer = $0a       ; destination address in VRAM
473
474         ; set CGDRAM destination address
475         lda DestPointer         ; get destination address
476         sta CGADD               ; set CGRAM destination address
477
478         ldy #$0000              ; set Y to zero, use it as loop counter and of
479 CGRAMLoop:
480         lda (SrcPointer, S), Y  ; get the color low byte
481         sta CGDATA              ; store it in CGRAM
482         iny                     ; increase counter/offset
483         lda (SrcPointer, S), Y  ; get the color high byte
484         sta CGDATA              ; store it in CGRAM
485         iny                     ; increase counter/offset
486         cpy NumBytes            ; check whether 32/$20 bytes were transfered
487         bcc CGRAMLoop           ; if not, continue loop
488
489         ; all done
490         pld                     ; restore caller's frame pointer
491         plx                     ; restore old stack pointer
492         rts
493 .endproc
494 ;-------------------------------------------------------------------------------
495
496 ;-------------------------------------------------------------------------------
497 ;   Copies the OAMRAM mirror into OAMRAM
498 ;-------------------------------------------------------------------------------
499 .proc   UpdateOAMRAM
500         phx                     ; save old stack pointer
501         ; create frame pointer
502         phd                     ; push Direct Register to stack
503         tsc                     ; transfer Stack to... (via Accumulator)
504         tcd                     ; ...Direct Register.
505         ; use constants to access arguments on stack with Direct Addressing
506         MirrorAddr  = $07       ; address of the mirror we want to copy
507
508         ; set up DMA channel 0 to transfer data to OAMRAM
509         lda #%00000010          ; set DMA channel 0
510         sta DMAP0
511         lda #$04                ; set destination to OAMDATA
512         sta BBAD0
513         ldx MirrorAddr          ; get address of OAMRAM mirror
```

```
514         stx A1T0L               ; set low and high byte of address
515         stz A1T0B               ; set bank to zero, since the mirror is in WRA
516         ldx #$0220              ; set the number of bytes to transfer
517         stx DAS0L
518
519         lda #$01                ; start DMA transfer
520         sta MDMAEN
521
522         ; OAMRAM update is done, restore frame and stack pointer
523         pld                     ; restore caller's frame pointer
524         plx                     ; restore old stack pointer
525         rts
526 .endproc
527 ;--------------------------------------------------------------------------
528
529 ;--------------------------------------------------------------------------
530 ;   Interrupt and Reset vectors for the 65816 CPU
531 ;--------------------------------------------------------------------------
532 .segment "VECTOR"
533 ; native mode   COP,        BRK,        ABT,
534 .addr           $0000,      $0000,      $0000
535 ;               NMI,        RST,        IRQ
536 .addr           NMIHandler, $0000,      $0000
537
538 .word           $0000, $0000    ; four unused bytes
539
540 ; emulation m.  COP,        BRK,        ABT,
541 .addr           $0000,      $0000,      $0000
542 ;               NMI,        RST,        IRQ
543 .addr           $0000,      ResetHandler, $0000
544 ;--------------------------------------------------------------------------
```

JoypadSprite.s hosted with ❤ by **GitHub** **view raw**

Boy, has our little demo grown. Packing 500+ lines of code into a single file is usually not a good idea. We'll alleviate this [next time][2] when we talk about how to modularize our code and make it easier to reuse and navigate. For now, let's look at the sections that changed.

Lines 1 through 204: There's little to no change here. We added a handful of new symbols, `HVBJOY`, `STDCNTRL1L` / `H`, `JOYPAD`, `JOYTRIGGER1`, and `JOYHELD1` are new symbols/addresses we'll use to read the joypad. The `*_BUTTON` constants are bit masks we'll use to detect which button was pressed. Next, we initialize all data and place our sprites at the center of the screen. Lines 199 and 200 are important. Polling the joypads can be turned off if

necessary, we must make sure that it is turned on. Otherwise, the SNES won't poll the controllers/joypads for new input.

With initialization out of the way, let's look at the updated main game loop.

Lines 214 through 230: Here's where the magic happens. The first thing we need to do is to make sure that the SNES has finished polling the joypad(s) for input. This is what lines 215 through 217 do. The SNES polls the joypads once each frame (at the beginning of V-Blank) and it takes a few cycles to finish (as the communication between console and joypad is serial, similar to the NES). Once the console has polled the joypads, we switch the size of register A to 16 bits. We read the input just polled from `STDCNTRL1`. We then use the Y register to load the input we polled *last* frame from `JOYPAD1`. We store the newly polled input in A in `JOYPAD1` (so it becomes the last frame's input on the next game loop iteration/V-Blank).

So after line 222, we have this frame's input in A, and the last frame's input in Y, and stored this frame's input in WRAM at address `JOYPAD1`. We need this to be able to differentiate which button has been pressed since the last frame and which has been held (if any). We do this by transferring the last frame's input from Y to A and exclusive `OR`'ing with the current frame's input. After that, A only holds the buttons that were pressed during this frame, but not last. Aka, these are the buttons the player has just pressed. We stored this information in `JOYTRIGGER1`.

We repeat this process in lines 228 through 230 to find all the buttons that have been held since the last frame and store that information in `JOYHELD1`.

Now we know which buttons the players have either newly pressed or held since the last frame. We are now ready to check whether the dpad was used and move the sprites accordingly. We'll check the dpad in the order of *up*, *down*, *left*, *right*.

Lines 234 through 258: Here's where even more of the magic happens. First, we reset the accumulator to zero. We want the sprites to move up whenever we pressed or held the up button on the dpad. To do this, we `OR` in all triggered and held buttons. Then we use a constant bit mask to check whether the bit of the up button set (aka, was pressed or held). If it wasn't pressed, we just jump ahead and check the next button.

If the up button was pressed or held, we next need to move the four sprites up. This code shouldn't hold any surprises by now. We use Y as the loop counter and X as the offset. We load the current vertical position from the OAM mirror and subtract the vertical movement speed. Then we check whether the sprites would move the upper boundary. If it does, we jump to `CorrectVerticalPositionDown` to move all sprites to the upper boundary.

Lines 260 through 290: This is almost completely similar to the previous section. Only one part is slightly different: We check whether the sprites could cross the lower boundary *before* we move. This is necessary because the lower two sprites could cross the boundary before the upper two. If that is the case, we jump to `CorrectVerticalPositionUp`.

Lines 292 through 310: These sections of code simply set the position of the sprites of the upper and lower boundary respectively. We use the constants defined at the start of the code for this.

Lines 312 through 386: Again, very similar to what we did for the up and down button, expect that we change or correct the horizontal position of the sprites.

And that's it for handling input. The rest of the code is an exact copy from the bounding sprite example.

Challenges

Here are a few ideas (and a few hints) for you to experiment with and test your understanding of the code:

- Can you change the speed with which the sprites move using the A and B buttons? You'll need to add bit masks for the A and B buttons.
- Accelerate the sprites if a dpad button is held long enough? A minimum and maximum speed and a frame counter may prove useful here.
- Deaccelerate the sprites if the dpad button is let go? A frame counter can also count down.

Conclusion

To correctly handle input on a SNES, we poll the joypad (the mouse is handled differently; that may become its own article in the future) and check with the help of logical operators whether a certain button was pressed or not. This hasn't changed a lot 30+ years later, as you will still encounter code along the lines of `if( Gamepad.XButton.pressed )` in modern game engines. The logic we employ here is very similar.

As mentioned in the beginning, our code example has grown quite a bit. If you feel that there's a lot of code repetition here, you're correct. The code here is purposefully kept very verbose to make it easy to follow. But that makes coding harder than it needs to be.

So next time, we'll be breaking this code into several files to make it easier to handle and reuse for future projects. Additionally, we'll upgrade the makefile found in the SNES Assembly

Adventure repository to a CMake project for more convenient development.

Thank you for reading and see you next time!

Links and References

- This great post on the SFC Development Wiki covers the same ground as this article.
- nesdoug also has a great article on controller handling as well.
- Find a lot of detailed information on automatic and manual input poll in nocash's (author of several excellent emulators!) great SNES reference. Bookmark this site.
- All sources and files for this series can be found in the SNES Assembly Adventure repository.

ALSO ON MACHINE CODE CONSTRUCTION YARD

Sub Attack Dev Blog 01: A New Vectrex Game

8 years ago · 2 comments

I do assembly programming on old machines and boat building.

SNES Assembly Adventure 01: Develop

8 years ago · 16 comments

I do assembly programming on old machines and boat building.

SNES Assembly Adventure 10: …

4 years ago · 5 comments

I do assembly programming on old machines and boat building.

SNES Assem Adventure 0

4 years ago · 1 c

I do assembly on old machin building.